

JARVIS / WORLD MODEL

Jarvis 如何建立并维护自己的世界模型

它不是一座等待检索的记忆仓库，

而是一份持续编译的当前工作世界。



世界模型 = 实体与关系 + 当前认知 + 时间化证据 + 行动状态

工作世界一直在变，记忆池却只会继续堆积

普通 RAG 能找到材料，但不能自动维护一个正确、当前、可追溯的现实。

RAG POOL



01 没有结构

同一个人、项目和群被拆散在材料里

02 不会更新

新进展只会新增，不会纠正旧结论

03 无法忘记

过期判断继续参与下一轮决策

CURRENT WORLD

需要维护的不是材料，
而是“现在应该如何理解世界”。

旧结论

重新判断

当前结论

检索回答“哪份材料相关”；世界模型回答“此刻什么是真的”。

世界模型不是一张表，而是四类语义共同成立



世界观则是 Principal 围绕当前目标，对这份世界模型形成的一次动态投影。

结构告诉 Jarvis“看谁”；证据和认知告诉它“现在该怎么判断”。

长期实体与实时行动，是两个不同的建模主体

把它们混成一张大表，会同时丢掉长期认知和执行过程。



世界模型同时回答：世界有什么，以及正在发生什么。

六类长期实体，围绕 Principal 形成工作坐标

实体不是固定字段模板，而是一页关于该对象的自然语言当前认知。



internal/domain/models.go

Principal 是坐标原点，不是普通实体节点；其它实体围绕它形成可理解的工作上下文。

同一个Jarvis 语境里，每个实体实际写什么

卡片展示的是 Summary 示例，不是固定表单字段。

Principal

我

长期目标：把 Jarvis 做成主动式数字分身

Person

王磊（示例）

角色：协作者；关系：参与 Jarvis

Project

Jarvis

目标：主动发现并完成有价值的工作

KeyMatter

讲清实体层与行动层

状态：正在完善世界模型示例

Group

Jarvis 项目群

用途：方案讨论、评审与反馈

Resource

《主动式数字分身》

类型：飞书 Wiki；归属：Jarvis

关系示例 Principal 主导 Project • Person 协作 Project • KeyMatter 属于 Project • Resource 承载项目资料

正在发生 Todo：发现图例缺少实例 ➡ Task：补齐世界模型例子 ➡ Run：生成并验收 Slides

抽象实体只有落到具体内容、归属与关系，才能成为 Agent 的判断坐标。

关系不是另一张图谱表，而是实体页中的可验证引用

可读的 Markdown 链接同时承担语义表达、目标校验和反向关系推导。

PROJECT SUMMARY

Project 「Jarvis」

负责人: [我](principal:1)
协作者: [王磊](person:12)
关键议题: [实体与行动模型](key_matter:7)
资料: [主动式数字分身](resource:31)

① PARSE

[文字](type:id)

从 Summary 提取实体 URI

② VALIDATE

目标必须真实存在

缺失引用直接报错，不静默吞掉

BACKLINKS

谁引用了这个实体？

Person / 王磊 → Project / Jarvis

KeyMatter / 实体与行动模型 → Project / Jarvis

扫描实体 Summary 派生，不维护第二份关系真源

⊗ 没有独立 RelationFact 表

同一个实体挂三种记录：现在、事实与认知历史

它们不是三份摘要，而是三种不同的时间语义。



internal/domain/progress.go

Fact 记录世界怎么变；PageRevision 记录 Jarvis 的理解怎么变。

行动状态把“变化”一路承接到“执行结果”

Todo、Task、ExecutionRun 是独立运行记录，不是实体 Summary 的字段。

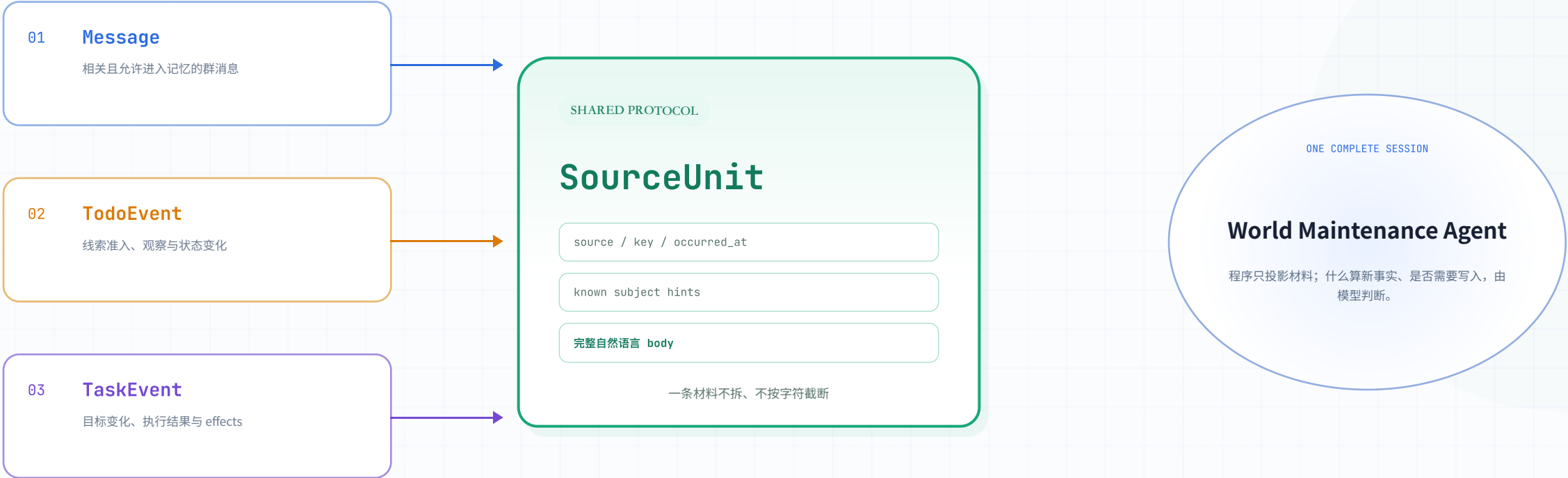
Project「Jarvis」 · KeyMatter「世界模型 Slides」 · Resource「目标 Wiki」



实体提供长期坐标；行动记录尚未闭环的现实，并把执行结果重新送回世界。

三类变化材料，被机械投影成同一种 SourceUnit

来源可以不同，但下游只接收一套自然语言证据协议。



核心方法是编译：先读当前世界，再决定它是否需要改变

FactEngine 不是摘要器，也不是任务执行器；它维护的是实体的当前长期认知。

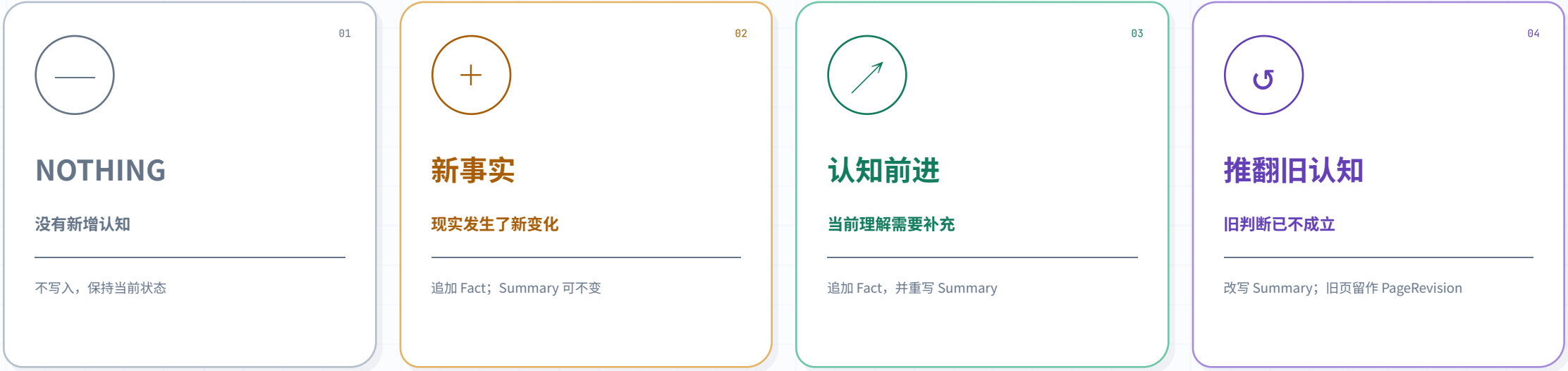


conf/prompts/fact-extract-system-prompt.md

编译不是把新材料拼上去，而是在完整旧认知上重新形成当前结论。

同一条新证据，可能产生四种完全不同的结果

分支由维护 Agent 根据上下文判断，不由 Go 枚举原因码。



世界没变

证据改变认知的程度

旧认知被纠正

语义判断保持开放；程序只保证事实追加、页面版本和写入边界。

五个硬边界，让认知可以重放、并发和纠错

程序不替模型判断语义，但必须保证材料不会被吞、旧页不会被乱覆盖。

01 独立游标

Message / TodoEvent / TaskEvent 各自推进

03 成功后推进

维护 Agent 失败时所有材料留待重放

事实批量写失败 → 整轮停止 → 不降级为单条偷偷续写



02 完整证据

超预算时减小行数，不截断 SourceUnit

04 CAS 写页

IfUnchangedSince 冲突后重读、重判

05 读回验证

确认真实当前页，不以工具返回成功代替

FactEngine 是自动主维护者；执行 Agent 只在拿到可靠新认知时补充

这是一套共同的世界模型工具，不是两条互相同步的专用数据管道。

定时触发

PRIMARY

FactEngine

持续消费增量证据

✓ 读实体页

✓ 比较变化

✓ 更新 Summary / Fact

✓ 不创建或执行 Task

同一世界模型
不是镜像同步

执行 / 主动巡检

AUXILIARY

M5 • Proactive

辅助补写者：只维护已查证的新事实

✓ 先完成真实目标

✓ 必要时修正实体页

✓ 外部动作仍必须成为 Task

✓ 不替 FactEngine 扫描全世界

list-pages

get-page

list-facts

update-page

一套通用工具边界

所有权按语义划分：FactEngine 维护认知，执行 Agent 完成目标并记录它真正改变了什么。

世界模型不会一次塞满上下文，而是按判断深度逐层展开

先给当前结论，需要追溯时再读完整页面、反向链接与历史 Fact。

M3 / ADMISSION

冻结一次上下文快照

Principal、选中 Project、群上下文、其它项目简报与资源提示

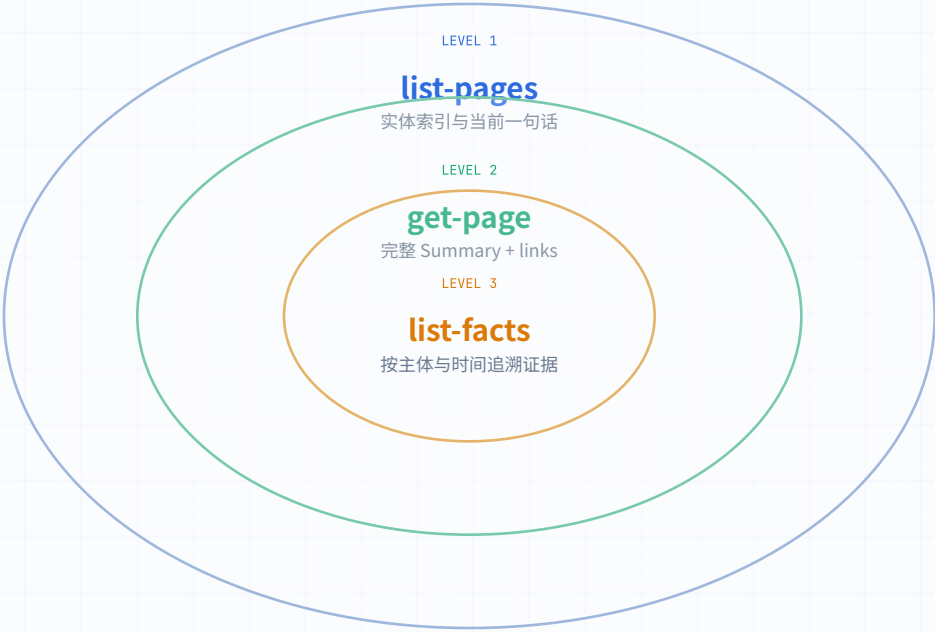
Todo.context_snapshot → Task.background

M5 / EXECUTION

读取新鲜行动状态

近期 Task brief + 未闭环 Todo brief，排除当前任务自身

current world 在执行时补充



少量默认上下文

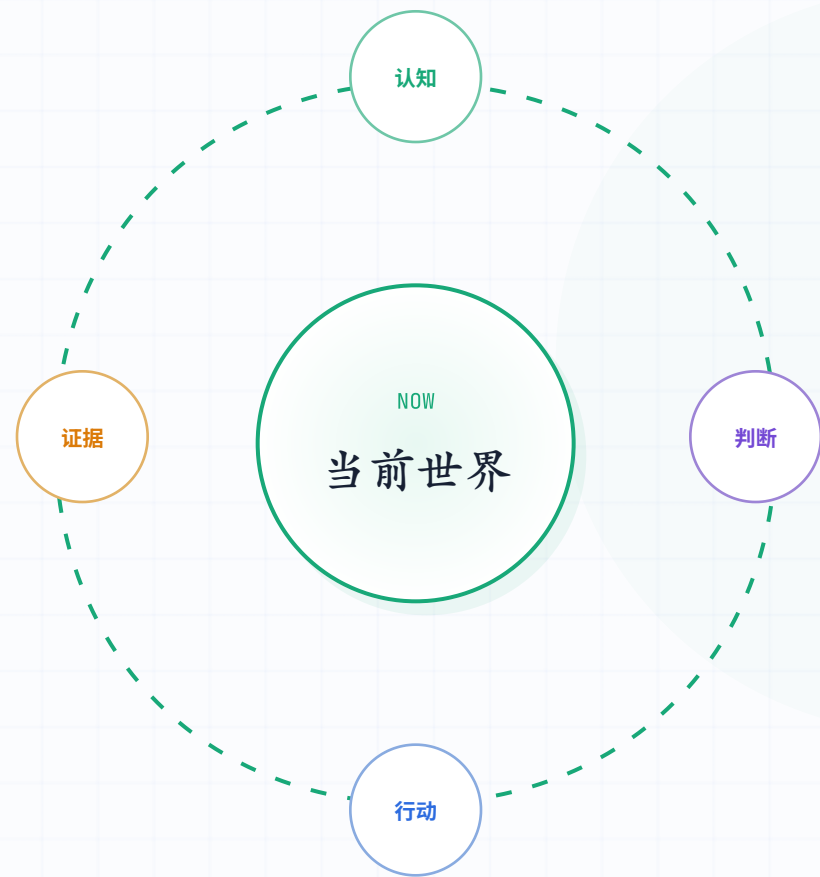
围绕当前目标主动下钻

得到足够判断证据后停止

THE OPERATING PRINCIPLE

Jarvis 不是在
~~检索记忆~~

而是在持续编译
下一次决策所需的现实



事实持续累积，认知持续修正，行动持续改变世界。